

Ray Tracing

Contents

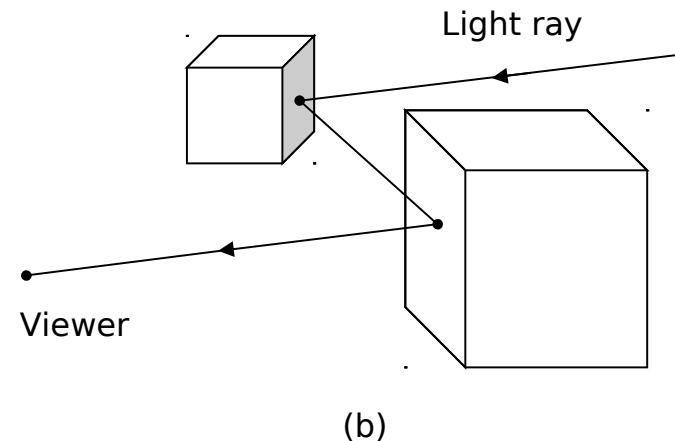
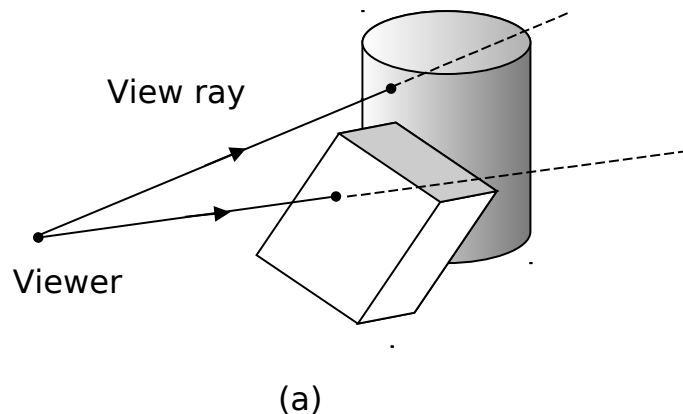
- Introduction
- Basic algorithm
- Recursive ray tracing
- Intersections
 - Ray-sphere
 - Ray-convex polyhedron
 - Ray-box
 - Ray-quadrics
 - Ray-any object
- Reflection and refraction
- New local reflection model
- Global reflection model
- Recursive computation
- Shadow computation
- Implementation

Introduction

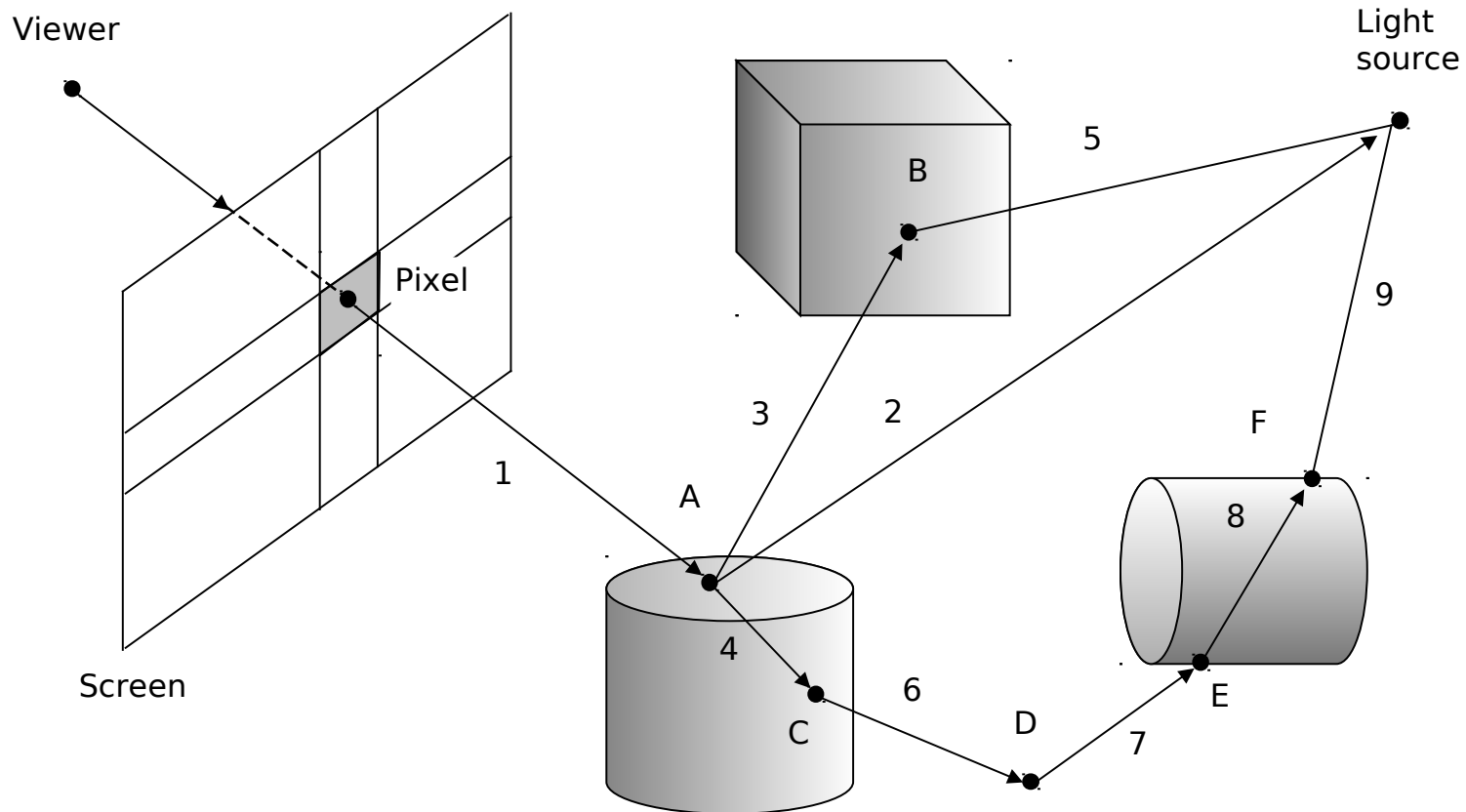
- ❑ Ray tracing was originally used in computer graphics by Appel in 1968
- ❑ In 1980, Turner Whitted has developed the first ray tracing model that integrates reflection, refraction, hidden surface removal, and shadow computation
- ❑ Partial solution to the global illumination problem
- ❑ (-) High processing time

Basic algorithm

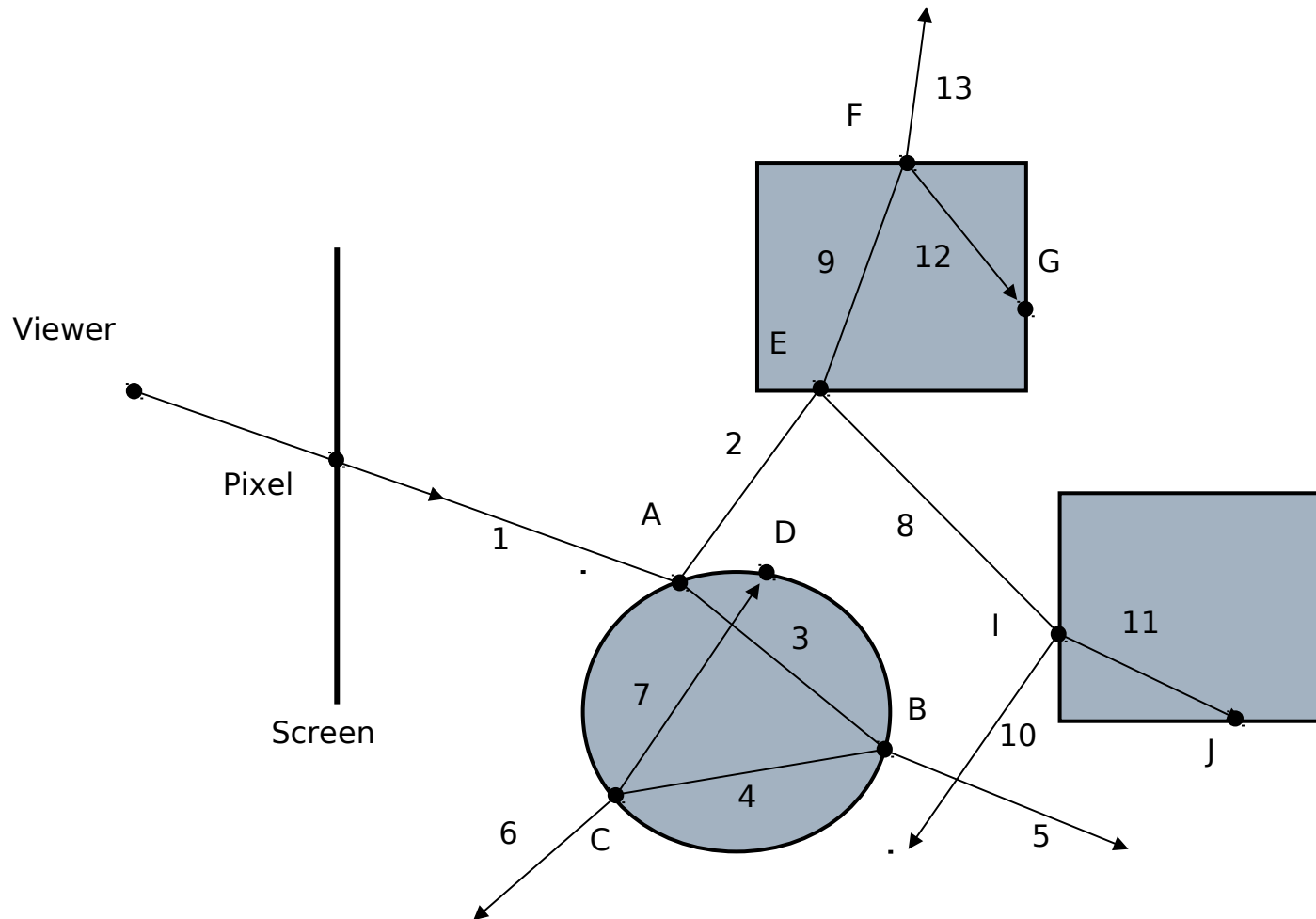
- ❑ Works completely in object space
- ❑ Color and intensity at a point are obtained by tracing a ray backwards from the eye through the point into the scene
- ❑ Illumination is a contribution of the direct illumination, reflected and transmitted rays



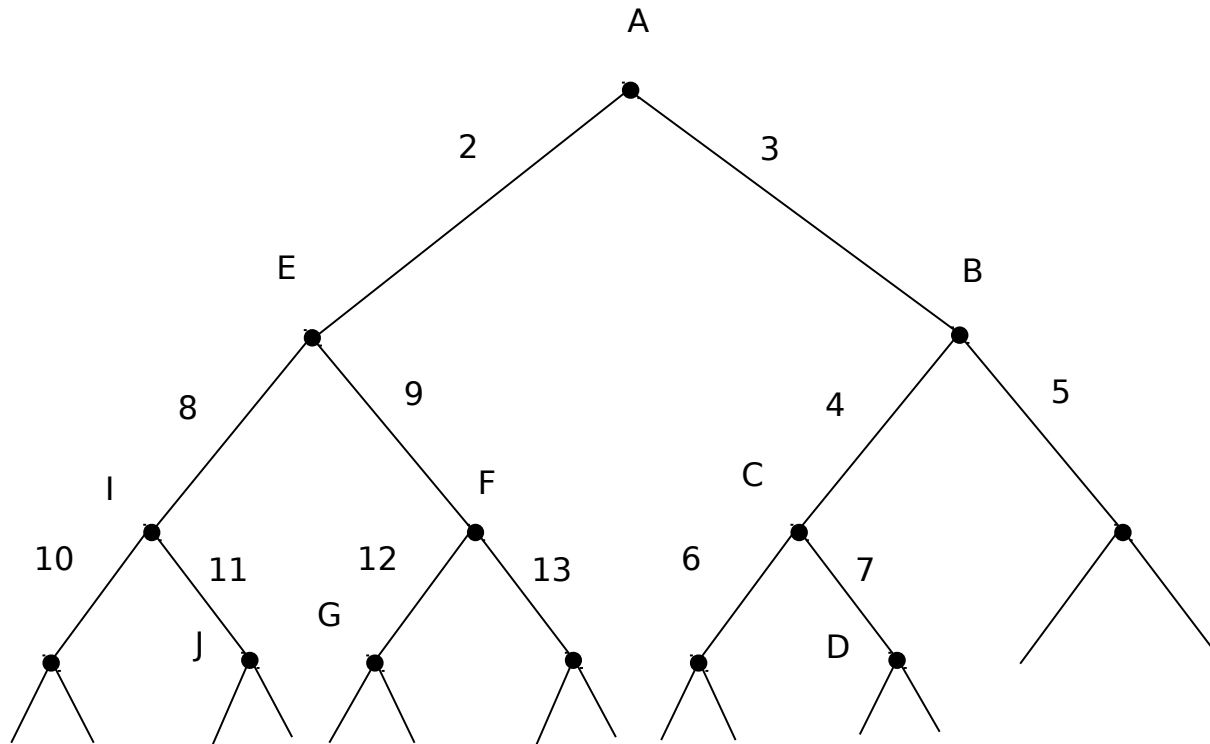
Basic algorithm



Recursive ray tracing



Recursive ray tracing - viewing ray tree



Simple recursive ray tracing procedure

TraseazaRaza (vectori start, vectori directie, int nivel, culori culoare)

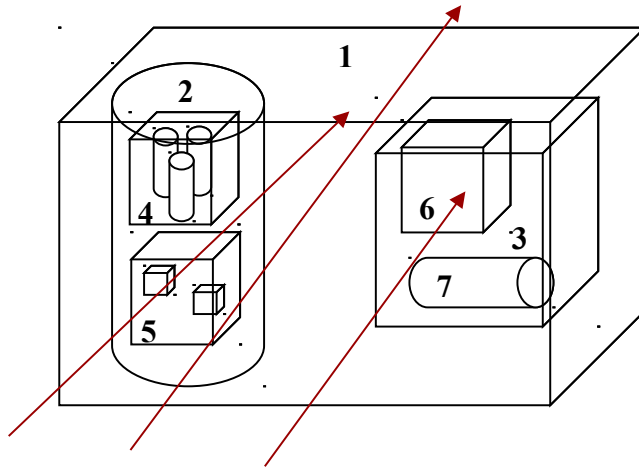
```
{  
    vectori    punct_intersectie, directie_reflexie, directie_refractie;  
    culori     culoare_locala, culoare_reflectata, culoare_refractata;  
  
    Intersectează raza cu toate obiectele și determină cel mai apropiat punct de  
    intersecție, dacă există, fata de începutul razei.  
  
    if (nu există intersecție)  
        culoare = culoarea de fond;  
    else {  
        culoare_locala = contribuția modelului local de culoare la punctul de  
        intersecție;  
        if ( nivel = nivel_maxim )  
            culoare_reflectata = black;  
            culoare_refractie = black;  
    }
```


Simple recursive ray tracing procedure

```
    else {
        Calculează direcția razei reflectate;
        TraseazaRaza(punct_intersectie,
                    directie_reflexie,
                    nivel+1,
                    culoare_reflectata);
        Calculează direcția razei refractate;
        TraseazaRaza(punct_intersectie,
                    directie_refractie,
                    nivel+1,
                    culoare_refractata);
    }
    Combina(        culoare, culoare_locala,
                    ponderea_locala_suprafata,
                    culoare_reflectata,
                    coeficient_reflexie_al_suprafetei,
                    culoare_refractata,
                    coeficient_refractie_al_suprafetei);
}
} // TraseazaRaza
```

Ray tracing geometry - Intersections

- ❑ Intersection computation – time consuming
- ❑ 95% of processing time (Whitted, 1980)
- ❑ Speeding up the intersection computation:
 - bounding volumes (Whitted 1980, Clark 1976)
 - Avoid extensive computation of the intersection outside of bounding boxes
 - Bounding boxes: sphere, cylinder, cube, parallelepiped, quadrics (i.e. ellipsoid, paraboloid, hyperboloid, etc)



Intersection: ray-sphere

View ray: $P1(x1, y1, z1)$ and $P2(x2, y2, z2)$

Parametrical equation of the view ray:

$$x = x1 + (x2 - x1) t = x1 + it$$

$$y = y1 + (y2 - y1) t = y1 + jt$$

$$z = z1 + (z2 - z1) t = z1 + kt$$

A sphere at centre (l, m, n) and radius r :

$$(x - l)^2 + (y - m)^2 + (z - n)^2 = r^2$$

Substituting for x, y and z :

$$at^2 + bt + c = 0$$

where:

$$a = i^2 + j^2 + k^2$$

$$b = 2i(x1 - l) + 2j(y1 - m) + 2k(z1 - n)$$

$$c = l^2 + m^2 + n^2 + x1^2 + y1^2 + z1^2 + 2(-l x1 - m y1 - n z1) - r^2$$

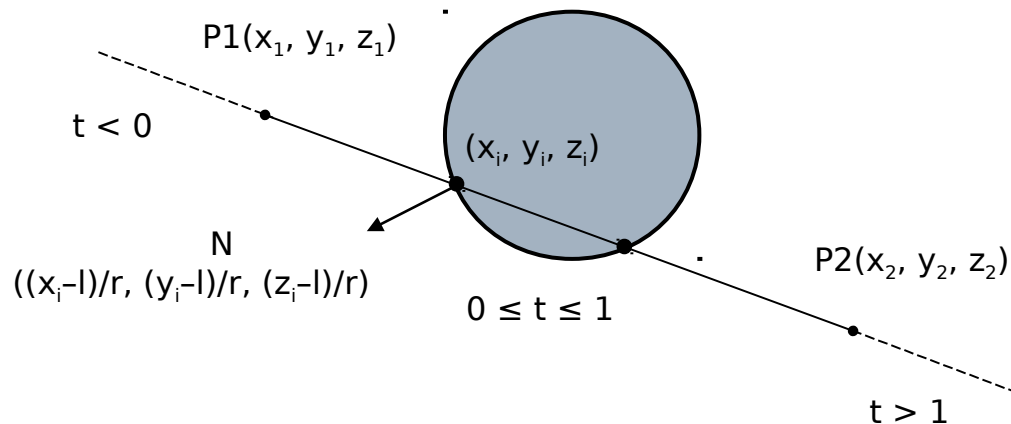
Intersection: ray-sphere

If $\Delta < 0$, the line does not intersect the sphere

$\Delta = 0$, the line is tangential to the sphere

$\Delta > 0$, there is a front and back intersections

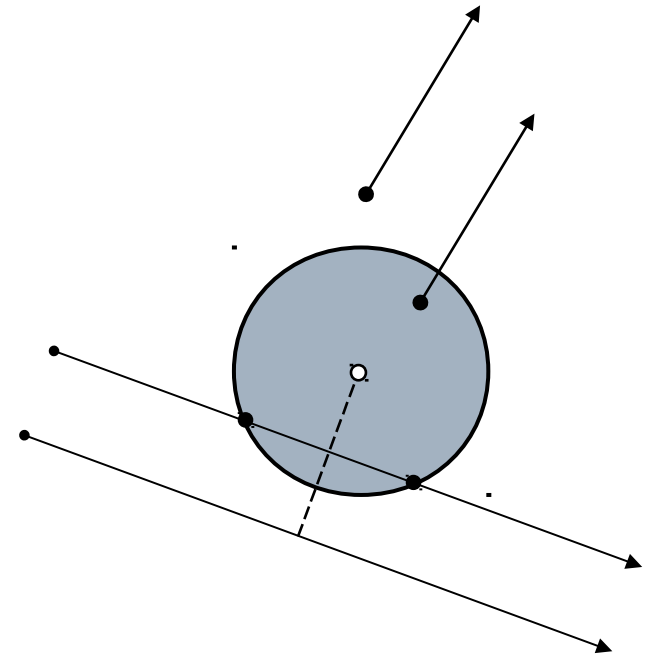
Value of t gives the position of the intersections relative to P1 and P2:



Intersection: ray-sphere

Speeding up the decision on the intersection:

1. Find if the ray's origin is outside the sphere
2. Find the closest approach of the ray to the sphere's centre
3. If the ray is outside and points away from the sphere, the ray must miss the sphere
4. Else, find the distance from the closest approach to the sphere surface
5. If the value is negative, the ray misses the sphere
6. Else, from above, find the ray-surface distance
7. Calculate the intersection coordinates
8. Calculate the normal at the intersection point



Intersection: ray-convex polyhedron

- ❑ Object - convex polygonal model
- ❑ Test the ray individually against each polygon:
 1. Determine the equation of the polygon plane
 2. Check for an intersection between this plane and the ray
 3. Check the intersection is inside the polygon

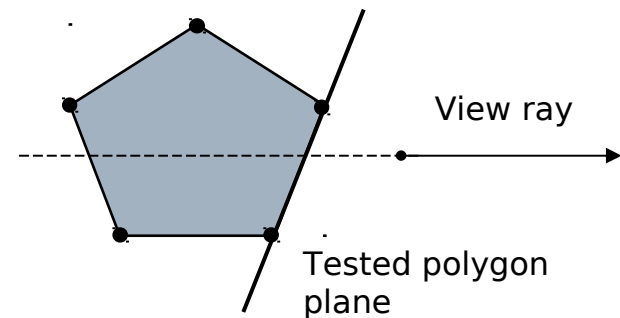
If the polygon plane equation is:

$$ax + by + cz + d = 0$$

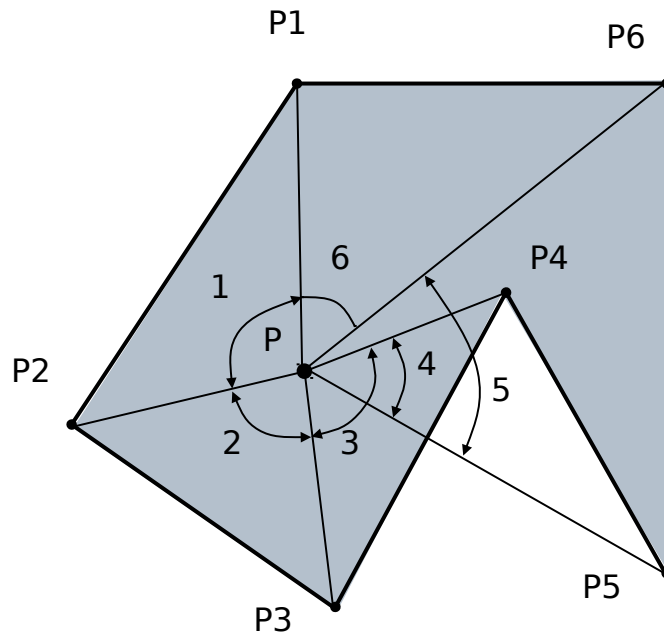
Then the intersection with a parametric line is given by t:

$$t = - (a x_1 + b y_1 + c z_1) / (a i + b j + c k)$$

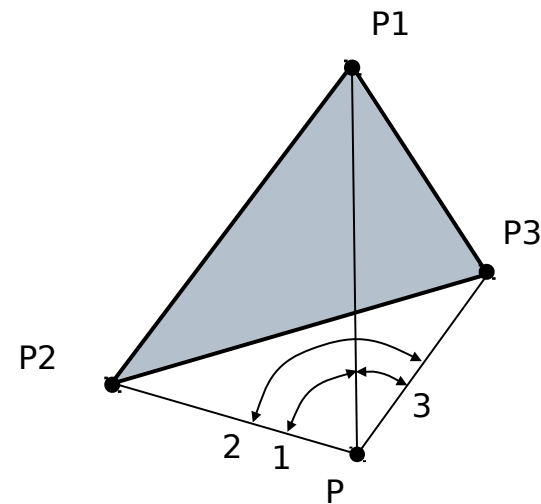
Exit the test if $t < 0$,
the intersection is on back side of the viewer.



Intersection: ray-convex polyhedron

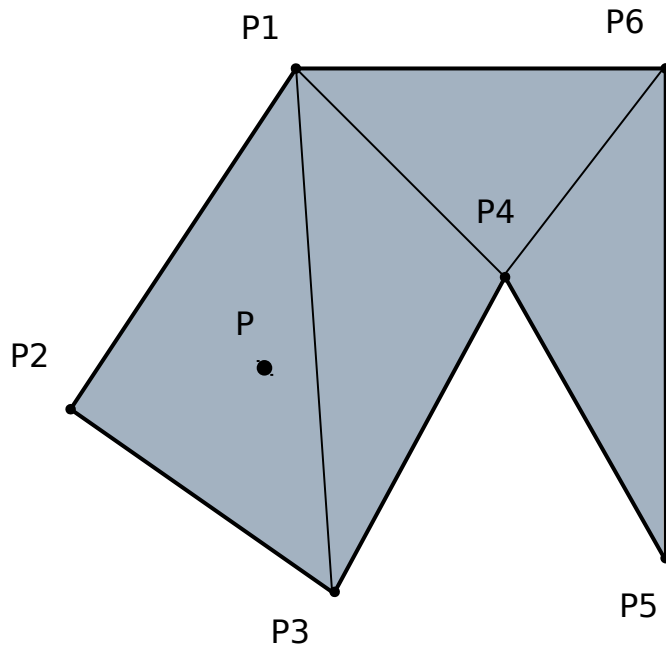


(a) $\angle (1 + 2 + 3 - 4 + 5 + 6) = 360^\circ$
 P inside of polygon ($P1, \dots, P6$)



(b) $\angle (1 - 2 + 3) = 0^\circ \neq 360^\circ$
 P outside of polygon ($P1, P2, P3$)

Intersection: ray-convex polyhedron



Triangulation:

$$P_1, \dots, P_6 \rightarrow \Delta P_1 P_2 P_3$$

$$\Delta P_1 P_3 P_4$$

$$\Delta P_4 P_5 P_6$$

$$\Delta P_1 P_4 P_6$$

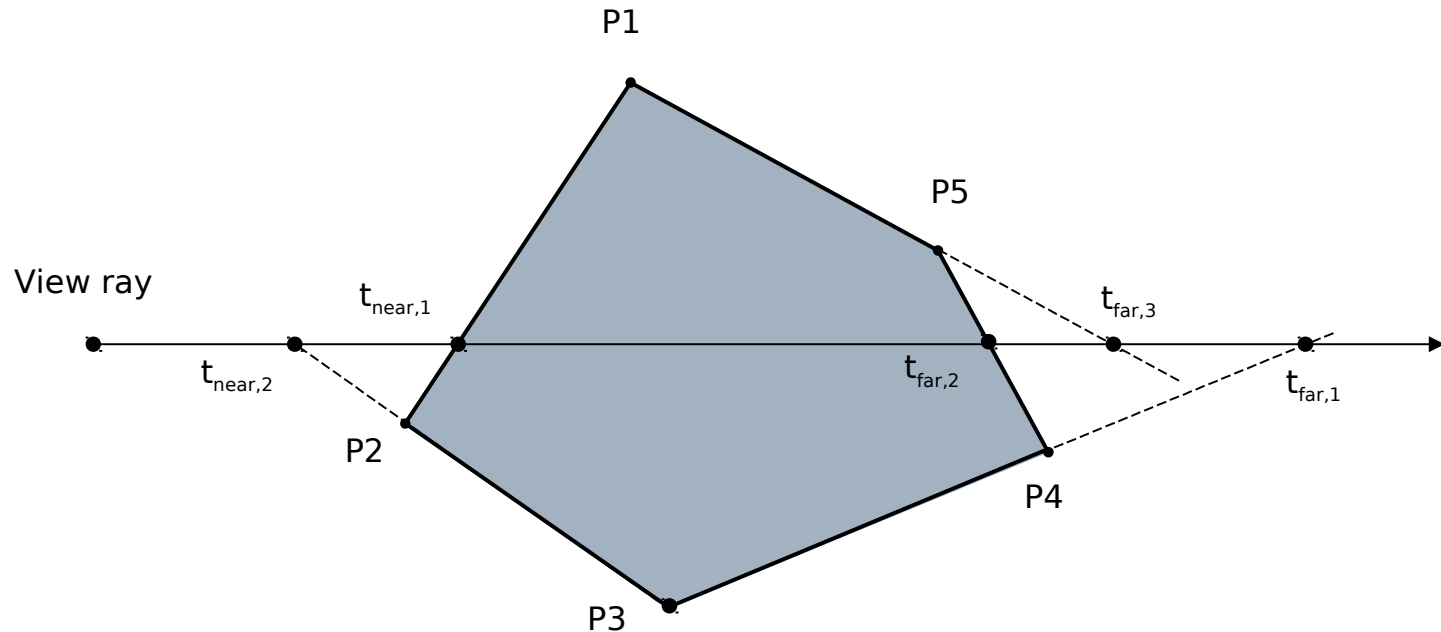
Check the position of P against a triangle:

$$P \in \Delta P_1 P_2 P_3$$

Conclusion:

$$P \in P_1, \dots, P_6$$

Intersection: ray-convex polyhedron



Cyrus-Beck algorithm:

Initialize t_{near} to large negative value, t_{far} to large positive value

If (plane is back-facing) and ($t < t_{far}$) then $t_{far} = t$

If (plane is front-facing) and ($t > t_{near}$) then $t_{near} = t$

If ($t_{near} > t_{far}$) then (exit - ray misses)

Intersection: ray-box

Bounded volume: (x_{e1}, y_{e1}, z_{e1}) and (x_{e2}, y_{e2}, z_{e2})

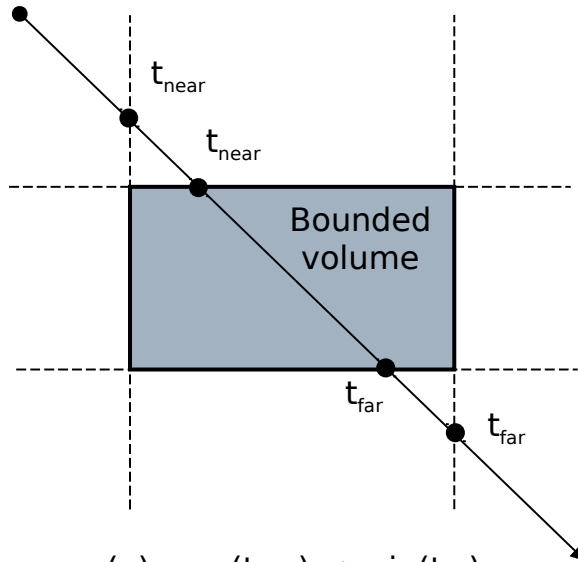
Intersections with the x axis:

$$t_{1x} = (x_{e1} - x_1) / (x_2 - x_1)$$

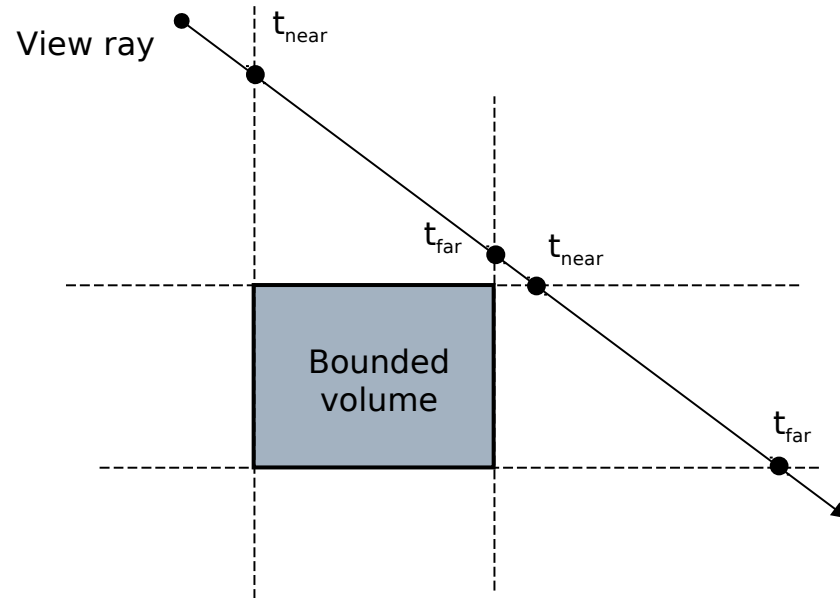
$$t_{2x} = (x_{e2} - x_1) / (x_2 - x_1)$$

Where x_1 and x_2 are the coordinates of the view ray end points

View ray



(a) $\max(t_{near}) \leq \min(t_{far})$



(b) $\max(t_{near}) > \min(t_{far})$

Intersection: ray-quadrics

- ❑ Quadrics: cylinder, ellipsoid, paraboloid, and hyperboloid
- ❑ General equation for a quadric:

$$Ax^2 + Ey^2 + Hz^2 + 2Bxy + 2Fyz + 2Cxz + 2Dx + 2Gy + 2Iz + J = 0$$

That is the following matrix based form:

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} A & B & C & D \\ B & E & F & G \\ C & F & H & I \\ D & G & I & J \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = 0$$

Using the same substitution as for the sphere the a, b and c coefficients are:

$$a = Ax_d^2 + Ey_d^2 + Hz_d^2 + 2Bx_dy_d + 2Cx_dz_d + 2Fy_dz_d$$

$$b = 2(Ax_1x_d + Ey_1y_d + Hz_1z_d + B(x_1y_d + x_dy_1) + C(x_1z_d + x_dz_1) + F(y_1z_d + y_dz_1) + Dx_d + Gy_d + Iz_d)$$

$$c = Ax_1^2 + Ey_1^2 + Hz_1^2 + 2Bx_1y_1 + 2Cx_1z_1 + 2Fy_1z_1 + 2Dx_1 + 2Gy_1 + 2Iz_1 + J$$

where (x_d, y_d, z_d) is the normalized vector of the viewing ray direction

Intersection: ray-quadrics

- Sphere

$$(x - l)^2 + (y - m)^2 + (z - n)^2 = r^2$$

where (l, m, n) is the centre of the sphere

- Infinite cylinder

$$(x - l)^2 + (y - m)^2 = r^2$$

- Ellipsoid

$$(x - l)^2 / \alpha^2 + (y - m)^2 / \beta^2 + (z - n)^2 / \gamma^2 - 1 = 0$$

where α , β and γ are the semi-axes

- Paraboloid

$$(x - l)^2 / \alpha^2 + (y - m)^2 / \beta^2 - z + n = 0$$

- Hyperboloid

$$(x - l)^2 / \alpha^2 + (y - m)^2 / \beta^2 - (z - n)^2 / \gamma^2 - 1 = 0$$

Intersection: ray-any object

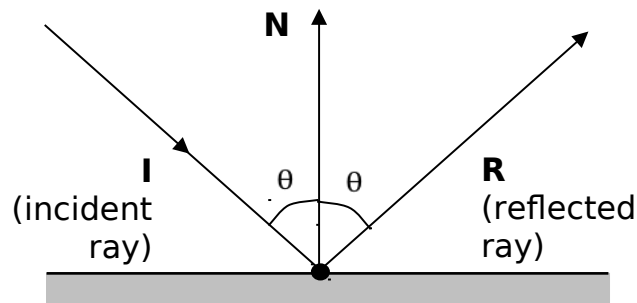
- ❑ Procedural modeling of the object – intersection depends on the type of the definition
- ❑ General approach:
 1. Use for each object a spherical bounded volume
 2. If the view ray intersects the sphere then the objects is subdivided in a set of object components. Each component has a spherical bounded volume
 3. The subdivision continues until the spherical bounded volume reach a minimum radius or the view ray does not intersect any more the spherical extension

Reflection and refraction

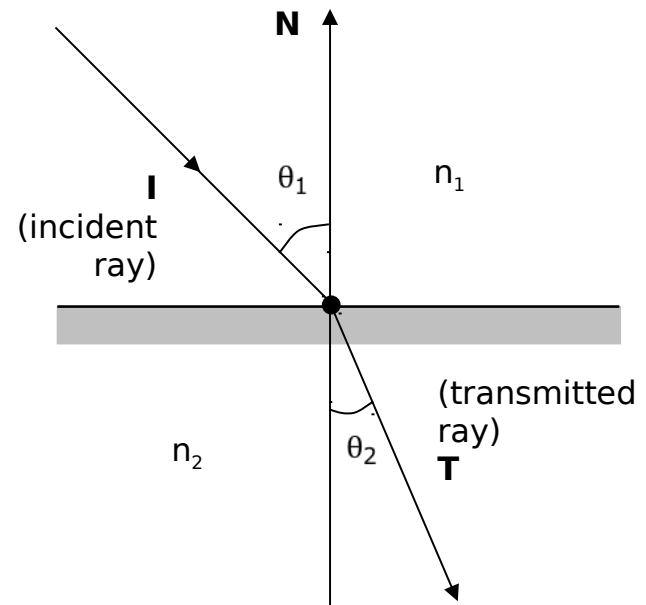
Snell's law: $\frac{\sin \theta_1}{\sin \theta_2} = \frac{n_2}{n_1}$ $n = \frac{n_2}{n_1}$

In 1984 Heckbert and Hanrahan determined the formula for the refracted (transmitted) ray:

$$\cos \theta_2 = n \left[1 - \frac{1}{n^2} (1 - \cos^2 \theta_1) \right]^{1/2}$$



(a) $R = I + 2N \cos \theta$

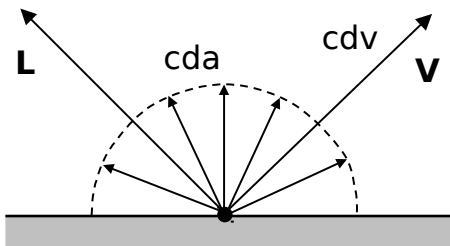


$$(b) \quad T = \frac{1}{n} I - \left(\cos \theta_2 - \frac{1}{n} \cos \theta_1 \right) N$$

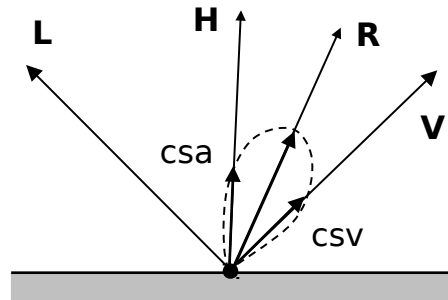
Illumination and reflection model

- Global illumination – direct, reflected and transmitted rays
- Direct illumination described by Phong model:

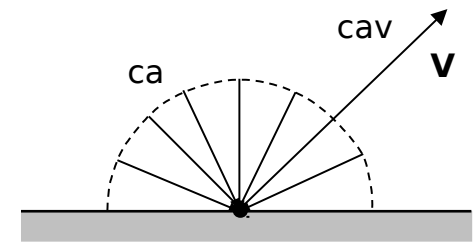
$$I_{\text{Phong}} = \text{ambient illumination} + \text{diffuse illumination} + \text{specular illumination}$$



(a) c_{dv} – diffuse component in direction V ; c_{da} – diffuse components in other directions.



(b) c_{sv} – specular component in direction V ; c_{sa} – specular components in other directions.



(c) c_{av} – ambient component in direction V ; c_a – ambient components from all directions

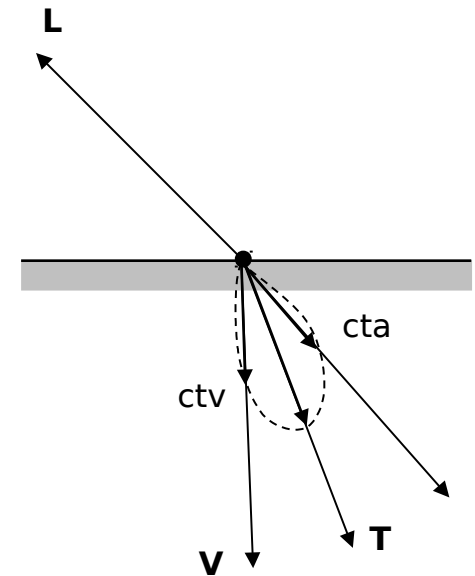
Illumination and reflection model

- Illumination for transparent objects
- Light that is seen through a transparent object due to a light source located at the other side of the object
- Generalizes the direct illumination component of the model

$$I_{\text{direct transmitted}} = k_t (N \cdot H')^n$$

where H' is the normal vector to a surface that refract the light to the V direction:

$$H' = (-L - nV) / (n - 1)$$



ctv – transmitted component in direction V ;
 cta – transmitted components in other directions.

New local reflection model

$$I_{\text{local}} = I_a k_a + I_i [k_d (N \cdot L) + k_s (N \cdot H)^{ns} + k_t (N \cdot H')^{nt}]$$

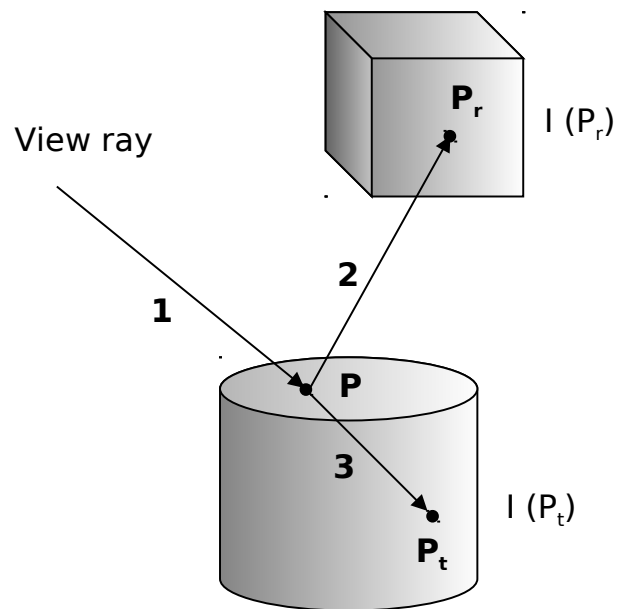
Global reflection model

$$I = I_{\text{local}} + k_{\text{rg}} I_{\text{reflection}} + k_{\text{tg}} I_{\text{transmitted}}$$

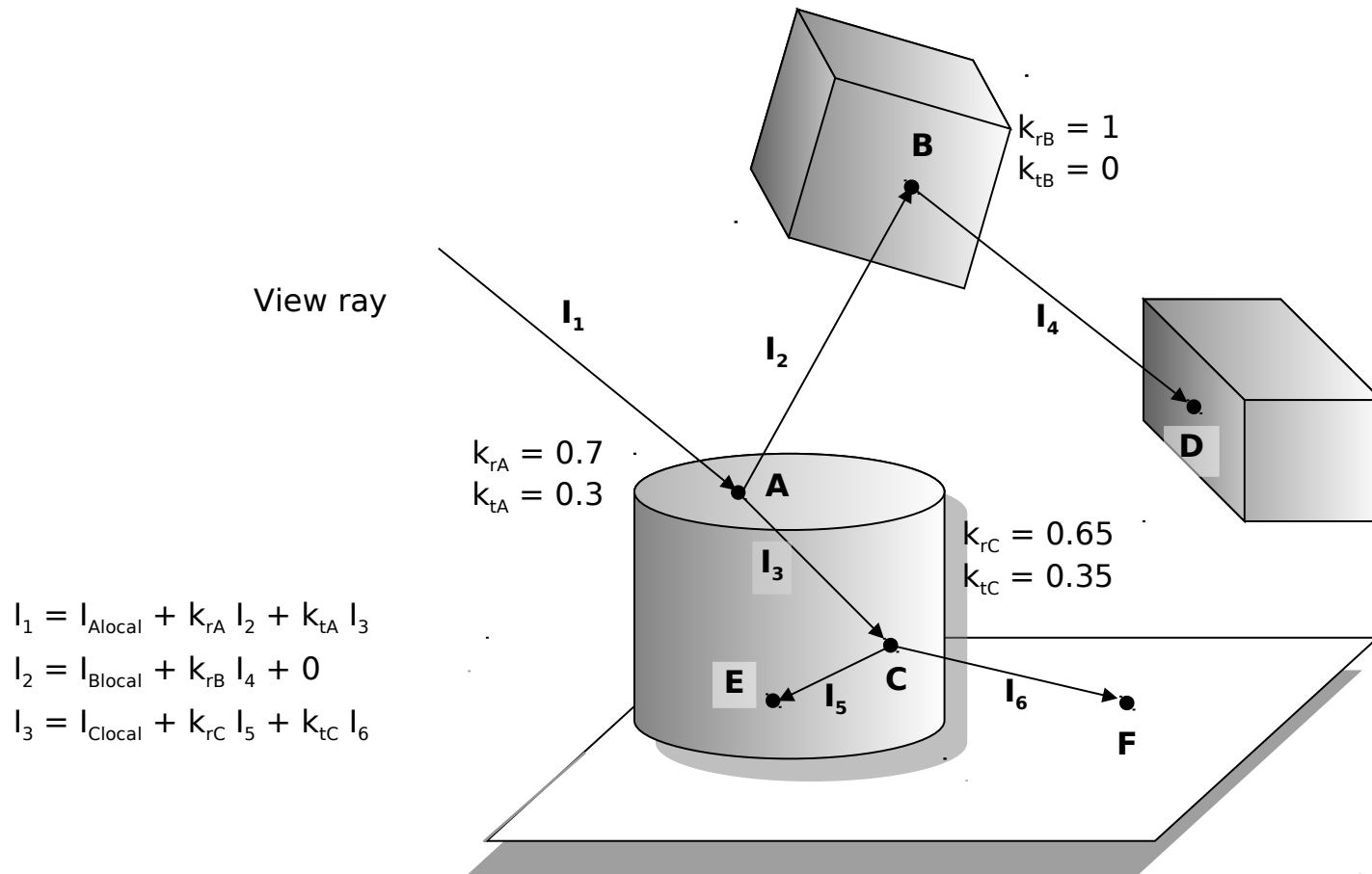
$$\begin{aligned} I &= I_{\text{local}} + I_{\text{global}} = \\ &= I_a k_a + I_i [k_d (N \cdot L) + k_s (N \cdot H)^n + k_t (N \cdot H')^n] + k_{\text{rg}} I_r + k_{\text{tg}} I_t \end{aligned}$$

Ray tracing - recursive computation

$$I(P) = I_{\text{local}}(P) + k_{\text{rg}} I(P_r) + k_{\text{tg}} I(P_t)$$



Ray tracing - recursive computation



Ray tracing – shadow computation

- ❑ Global reflection model incorporates the shadow computation
- ❑ At each point computes the local illumination I_{local}
- ❑ If the point is not direct illuminated it is in shadow
- ❑ Shadow feeler – view ray from the point to the light source, whether another object lies in the path of L direction and obstructs the direct illumination of the point
- ❑ If the obstruction object is opaque the I_{local} reduces to ambient component
- ❑ In simple ray tracing approach the transmitted shadow feeder is not considered at all

Implementation

- ❑ Adaptive depth control
- ❑ Bounding volume extensions
- ❑ Precomputing the first intersections
- ❑ Spatial coherence
- ❑ Auxiliary data models
 - Octree
 - SEADS
 - BSP

Adaptive recursion level

- In 1983, Hall and Greenberg proposed to adapt the recursion level according with the properties of the materials the light interacts with
- Rays are attenuated in various ways as they pass through the scene
 - The reflected ray is attenuated by the global specular reflection coefficient (k_{rg})
 - The refracted ray is attenuated by the global transmission coefficient (k_{tg})
 - The ray through several intersections is attenuated by the product of these coefficients (e.g. $k_{rD} k_{rB} k_{rA}$)
 - If the product of the coefficients is below some threshold the contribution of the ray is below a significant value -> there will be no point in ray tracing back further than that level (e.g. ray 4, further than D)
- For RGB color system there are three color contributions

Adaptive recursion level - algorithm

```
TraseazaRaza (vectori start, vectori directie, int nivel, culori culoare
               float pondere_cumulata)
{
    vectori    punct_intersectie, directie_reflexie, directie_refractie;
    culori     culoare_locala, culoare_reflectata, culoare_refractata;

    if (pondere_cumulata < prag_pondere)
        culoare = black;

    else
        if ( nivel > nivel_maxim )
            culoare = black;
        else {
            Intersectează raza cu toate obiectele și determină cel
            mai apropiat punct de intersecție, dacă există, fata de începutul razei.

            if (nu există intersecție)
                culoare = culoarea de fond;
            else {
                culoare_locala = contribuția modelului local de culoare la
                               punctul de intersecție;
            }
        }
}
```


Adaptive recursion level - algorithm

```
    Calculează direcția razei reflectate;  
    TraseazaRaza(punct_intersectie,  
                directie_reflexie,  
                nivel+1,  
                culoare_reflectata,  
                pondere_cumulata * coeficient_reflexie_al_suprafetei);  
    Calculează direcția razei refractate;  
    TraseazaRaza(punct_intersectie,  
                directie_refractie,  
                nivel+1,  
                culoare_refractata  
                pondere_cumulata * coeficient_refractie_al_suprafetei);  
    Combina(culoare, culoare_locala,  
            ponderea_locala_suprafata,  
            culoare_reflectata,  
            coeficient_reflexie_al_suprafetei,  
            culoare_refractata,  
            coeficient_refractie_al_suprafetei);  
    }  
}  
} // TraseazaRaza
```

Bounding volume extensions

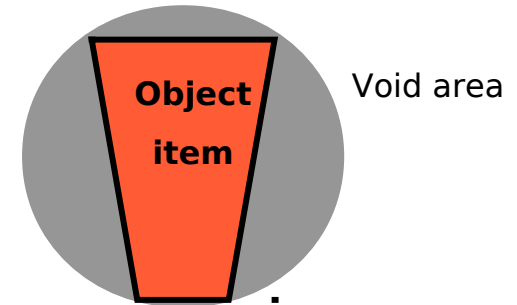
- Bounding volumes – intersection tests
- In 1984, Weghorst, Hooper și Greenberg
 - Defined a “void” area of a bounding volume – difference in area between the orthogonal projections of the object and bounding volume onto a plane perpendicular to the ray
 - Defined a cost function for the intersection test

Cost function:

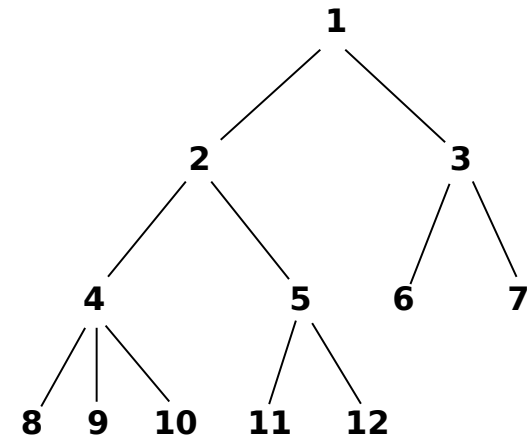
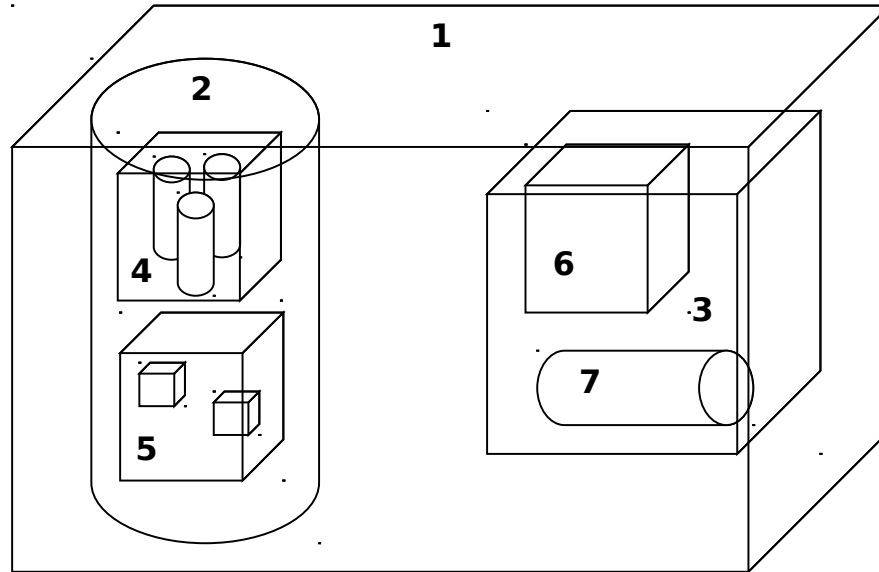
$$T = bB + il$$

where:

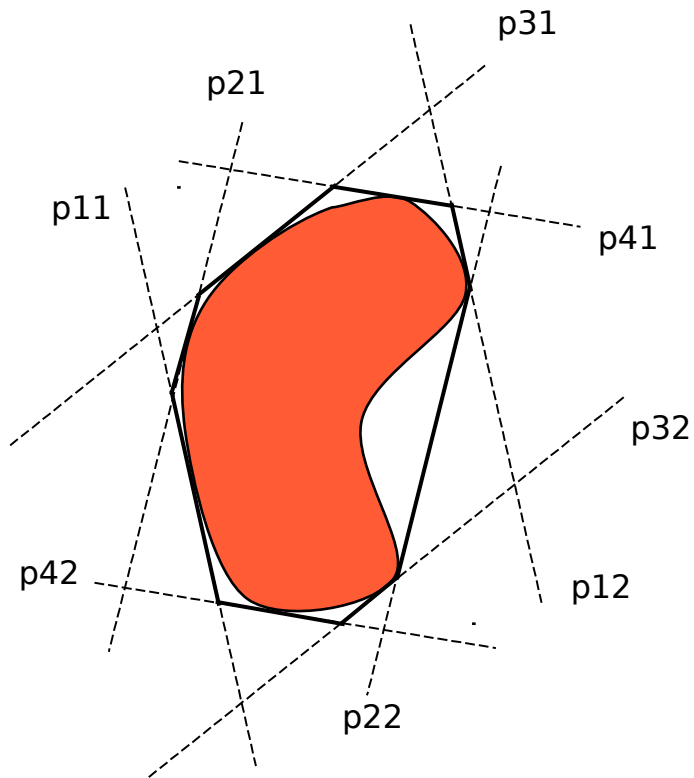
- T - total cost function
- b - number of intersection tests for the bounding volume
- B - cost of an intersection test for the bounding volume
- i - number of times the included item is tested for intersection ($i \leq b$)
- l - cost of the intersection test for an included item



Bounding volume - Hierarchical structure



Bounding volume - Convex hull



- Bounding volumes made from polyhedra consisting of pairs of parallel planes:
 $(p_{11}, p_{12}),$
 $(p_{21}, p_{22}),$
 $(p_{31}, p_{32}),$
 (p_{41}, p_{42})
- Simple computation of intersections

Bounding volume extensions

❑ Disadvantages:

1. Ray tracing efficiency depends on how well the objects fill the space of the bounding volumes
2. Each ray must be tested against the bounding extent of every object and the search time is a function of the scene complexity
3. Computation of an appropriate hierarchical structure of bounding volumes is difficult

Precomputing the first intersections

- Even for highly reflective scenes the average depth of ray tracing is 1 and 2
- In 1984, Weghorst, Hooper and Greenberg developed a precomputed ray tracer:
 - Intersection of the initial ray is evaluated in a preprocessing phase
 - Use for the first intersection computation a hidden surface algorithm (more efficient than the ray tracing computation)
 - A modified Z-Buffer algorithm computes for each pixel in the image plane, a pointer to the object visible at that pixel

Spatial coherence

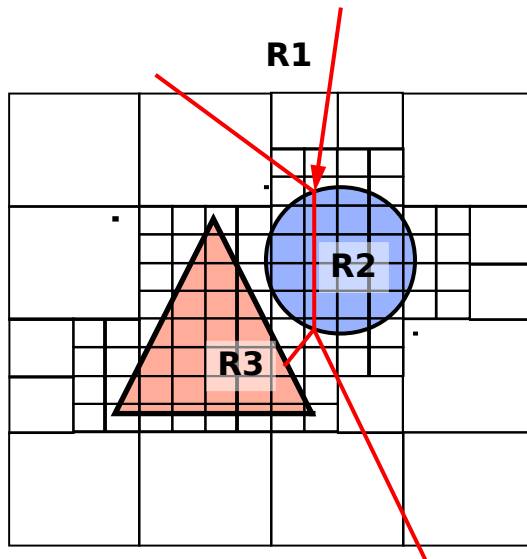
- Glassner (1984), Kaplan (1985), and Fujimoto, Tanaka and Iwata (1986)
- Space occupied by the scene is subdivided into regions
- Rather than checking a ray against all objects or sets of bounded objects, test just the region occupied by any objects
- Spatial coherence, spatial subdivision, space tracing
- Approach:
 1. Preprocess the space to set up an auxiliary data structure that contains information about the object occupancy of the space
 2. Rays are then traced using this auxiliary data structure to enter the object data structure
- Main advantages against the bounding volume extension:
 - Makes the rendering time constant (for a given resolution)
 - Eliminates the dependency on the scene complexity
- Octree representation, BSP, and SEADS

Ray tracer - Kaplan's requirements

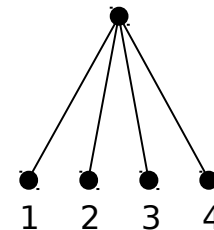
1. Computation time should be relatively independent of scene complexity
2. Per ray time should be relatively constant, and not depend on the origin or direction of the ray
3. Computation time should be in an acceptable range (e.g. one hour, few minutes)
4. The algorithm should not require the user to supply hierarchical object description or object clustering information
5. The algorithm should deal with a wide variety of primitive geometric types and easily extensible to new types
6. The use of coherence should not reduce the applicability to parallel processing or other advanced architectures

Octree representation in ray tracing

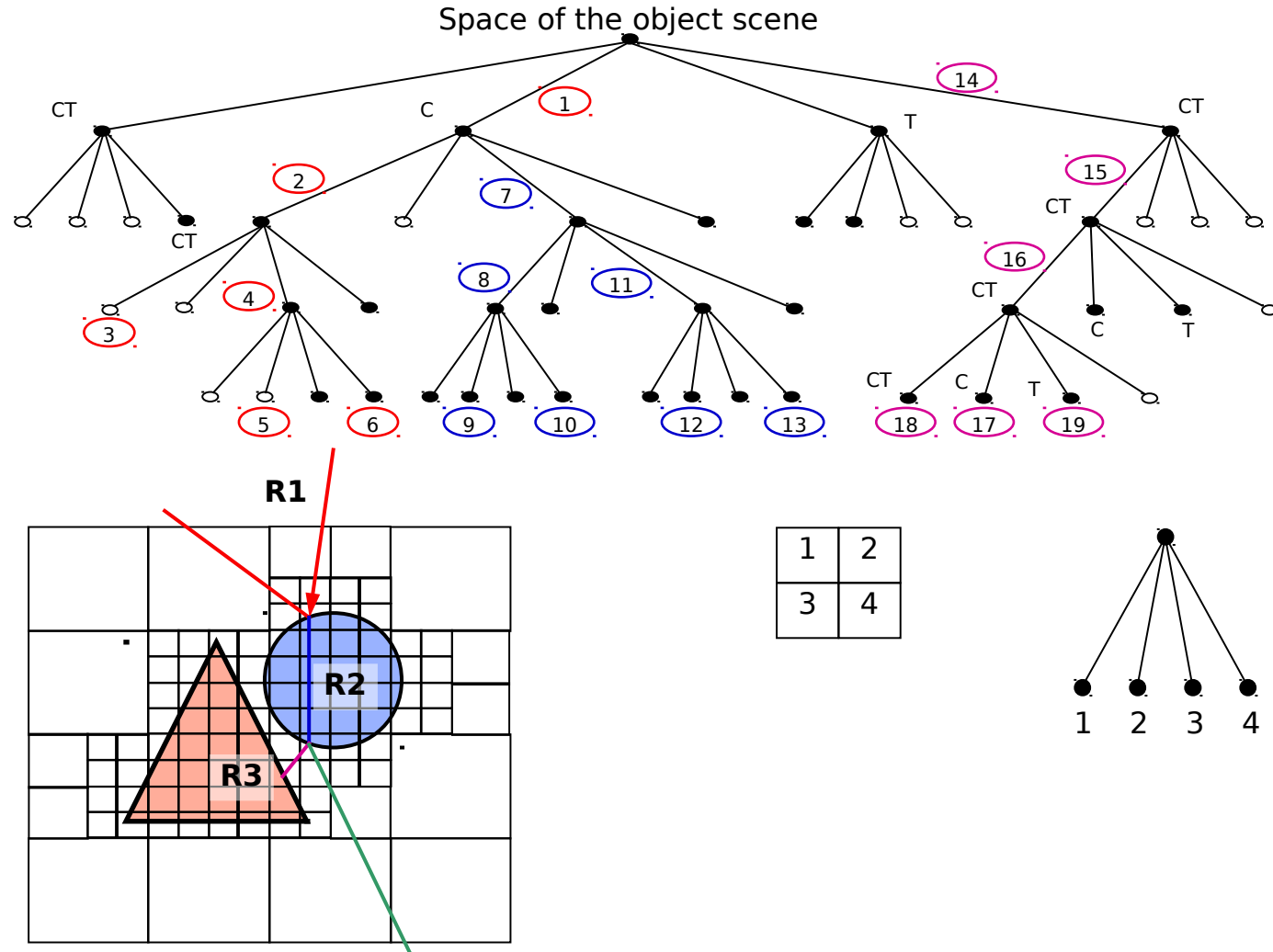
- Use spatial coherence
- Instead of doing intersection computation between the ray and every object in the scene, the algorithm traces the ray from sub-region to sub-region in the subdivision of occupied space
- Rapidly find the node in the octree that corresponds to a sub-region that a ray is passing through, and immediately accesses the objects that are on, or close to, the path of the ray



1	2
3	4

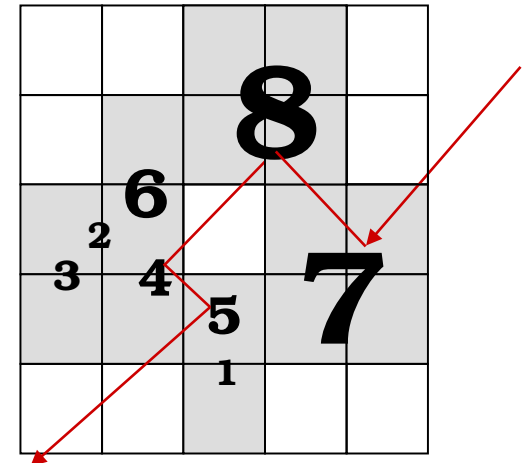


Octree representation in ray tracing



SEADS in ray tracing

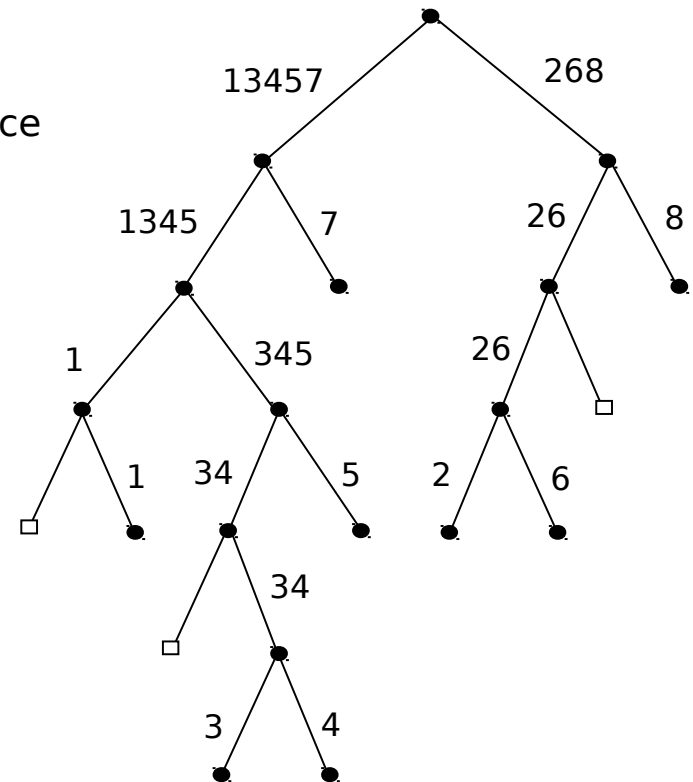
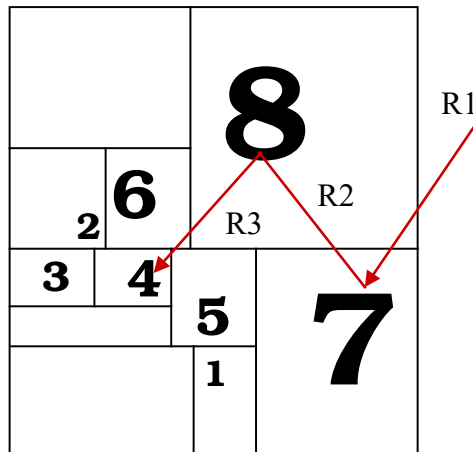
- ❑ SEADS (Spatially Enumerated Auxiliary Data Structure)
- ❑ Proposed by Fujimoto, Tanaka and Iwata (1986)
- ❑ Divide all the occupied space into equally sized voxels regardless of occupancy by objects
- ❑ Enables very fast tracking of rays from region to region
- ❑ Tracking algorithm: extension to 3D of the DDA (Digital Differential Analyzer) algorithm - used in 2D graphics to select a subset of the pixels passed through by a line
- ❑ 3D-DDA does not involve floating point multiplication and division, just addition, subtraction and comparison (integer based)
- ❑ The practice combines SEADS with Octree representation



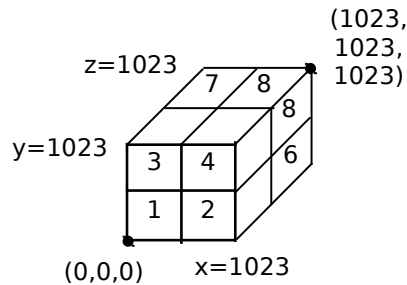
		8	8	
	6	8	8	
23	246		7	7
3	4	15	7	7
		1	8	

BSP in ray tracing

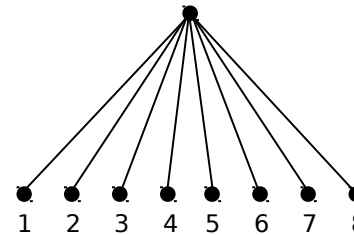
- ❑ BSP (Binary Space Partitioning)
- ❑ Proposed by Fuchs (1980), and used by Kaplan (1985)
- ❑ Divide the object space in two subspaces (e.g. parallel with coordinate system planes)
- ❑ Division plane: front-face space, back-face space
- ❑ Binary tree



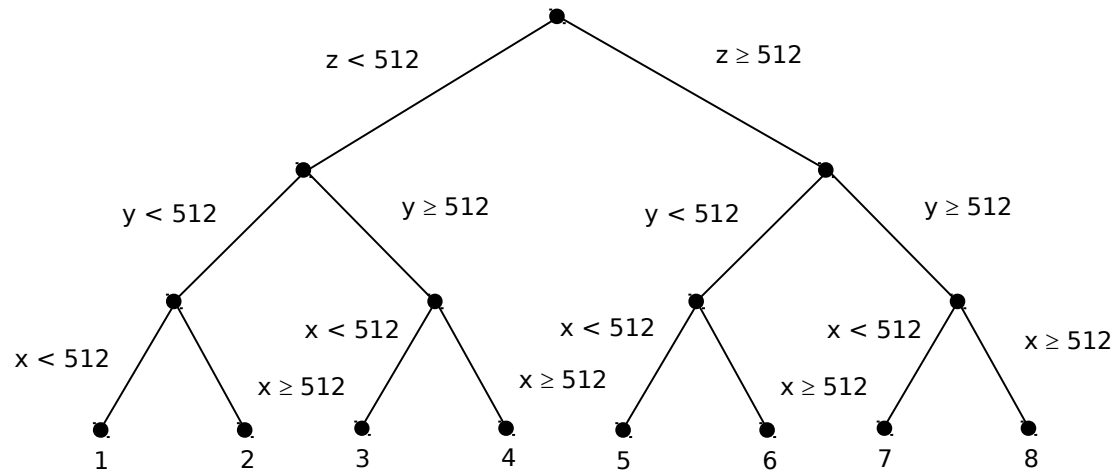
BSP in ray tracing - example



(a) BSP in 3D



(b) Octree



(c) BSP tree

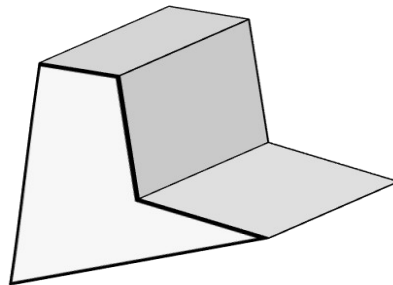
Questions and problems

1. What is the reason to stop the reflection and the refraction of the viewing ray?
2. Why a point in the scene is illuminated even it is not visible by the light source (direct illuminated)?
3. What parameters describes in the ray tracing process the properties of the following objects: a) cube of glass; b) cube of chalk; c) cube of shiny steel; d) cube of varnished wood?
4. Why the ray tracing simulates the illumination within the real world, even it uses the abstract concept of the viewing ray?
5. How the ray tracing method can be parallelized?
6. Explain the possibility of combining in ray tracing the computations of illumination and the texture mapping.

Questions and problems

7. Explain why in computation of intersections the classification of cases speeds up the rendering? Exemplify the case of a sphere.
8. Exemplify and explain the ray tracing algorithm for a scene of two cubes considering the following conditions: 2 point light sources; 4x4 pixels screen resolution; one cube is of transparent glass; one cube is of shiny steel.
9. Exemplify and explain the computation of intersection by the Cyrus-Beck algorithm for the object in the figure 1.

Figure
1



Questions and problems

10. Why in the new local reflection model at least one of the specular and transmitted components could be not null?
11. Exemplify and explain the using of an efficient bounding box system for the object in figure 1. Is a single box efficient enough?
12. Exemplify and explain the use of the SEADS structure for ray tracing of the object in figure 1.